DBMS PROBLEMS

MODULE 2

1:

Perform (i) Student U instructor (ii) Student ∩ Instructor
(iii) Student – Instructor (iv) Instructor – Student on the following tables:

Student

Fname	Lname
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

Instructor

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Mao
Francis	Johnson
Ramesh	Shah

Soln 1>

Given tables...

Student

fname	Lname
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

Instructor

fname	Lname
John	Smith
Ricardo	Browne
Susan	Mao
Francis	Johnson
Ramesh	Shah

i> Student ∪ Instructor

fname	Lname
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert
John	Smith
Ricardo	Browne
Susan	Mao
Francis	Johnson
Ramesh	Shah

ii> Student ∩ Instructor

fname	Lname
Ramesh	Shah

iii> Student − Instructor (Returns tuples which are in A but not in B)

fname	Lname
Susan	Yao
Johny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

iv> Instructor − Student (Returns tuples which are in Inst but not Students)

fname	Lname
John	Smith
Ricardo	Browne
Susan	Mao
Francis	Johnson

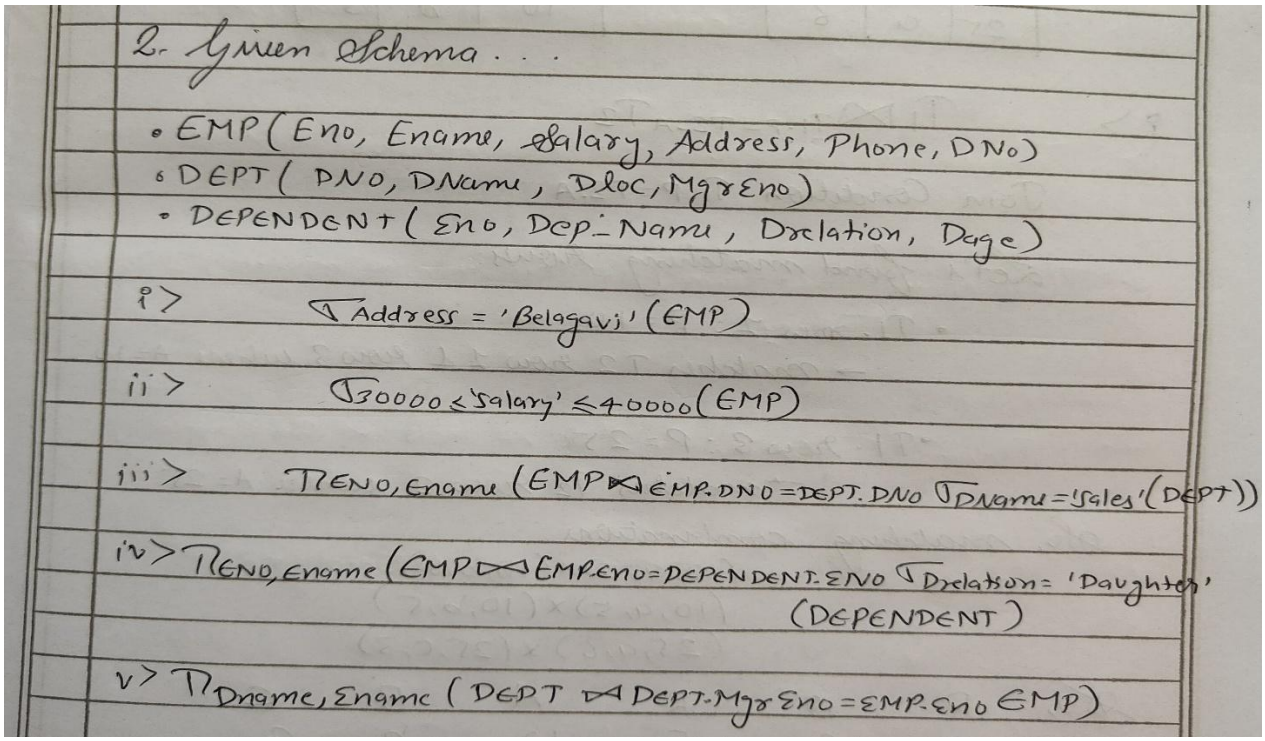
2.

Consider the following relational database schema and write the queries in relational algebra expressions:

EMP(Eno, Ename, Salary, Address, Phone, DNo)
DEPT(DNo, Dname, DLoc, MgrEno)
DEPENDENT(Eno, Dep_Name, Drelation, Dage)

(i) List all the employees who reside in 'Belagavi'.
(ii) List all the employees who earn salary between 30000 and 40000
(iii) List all the employees who work for the 'Sales' department
(iv) List all the employees who have at least one daughter
(v) List the department names along with the names of the managers

Ans:



3.

Consider the two tables T_1 and T_2 shown below:

T_1

P	Q	R
10	a	5
15	b	8
25	a	6

T_2

A	B	C
10	b	6
25	c	3
10	b	5

Show the results of the following operations:

(i) $T_1 \bowtie_{T_1.P=T_2.A} T_2$

(ii) $T_1 \bowtie_{T_1.Q=T_2.B} T_2$

(iii) $T_1 \bowtie_{(T_1.P=T_2.A \text{ AND } T_1.R=T_2.C)} T_2$

Ans:

3. Given...

T_1

P	Q	R
10	a	5
15	b	8
25	a	6

T_2

A	B	C
10	b	6
25	c	3
10	b	5

i) $T1 \bowtie_{T1.P=T2.A} T2$

Join Condition $T1.P=T2.A$

Let's find matching rows:

- $T1$. row 1: $P=10$
  - → matches $T2$ row 1 & row 3 where $A=10$
- $T1$. row 3: $P=25$
  - → matches $T2$ row 2 where $A=25$

So, matching combinations:

(10, a, 5) × (10, b, 6)

(10, a, 5) × (10, b, 5)

(25, a, 6) × (25, c, 3)

So,

P	Q	R	A	B	C
10	a	5	10	b	6
10	a	5	10	b	5
25	a	6	25	c	3

ii) $T1 \bowtie T1.Q = T2.B\ T2$

Join condition: $T1.Q = T2.B$

Let's find matches:

- T_1 row 2: $Q = b$
  → matches $T2$ row 1 & 3: $B = b$

So, combinations...

$(15, b, 8) \times (10, b, 6)$

$(15, b, 8) \times (10, b, 5)$

Result:

P	Q	R	A	B	C
15	b	8	10	b	c
15	b	8	10	b	5

iii) $T1 \bowtie T1.P = T2.A$ AND $T1.R = T2.C\ T2$

Check Matches: • $T_1(10, a, 5) \rightarrow T_2(10, b, 5)$

$T_1(25, a, 6) \rightarrow$ no match

$T2(15, b, 8) \rightarrow$ no match

So,

P	Q	R	A	B	C
10	a	5	10	b	5

Teacher's Signature

4.

Given the relational tables:

Employee:

EID	Name	DepID	Salary
1	Alice	10	5000
2	Bob	20	6000
3	Eve	20	6500

Department:

DeptID	DeptName
10	HR
20	IT
30	Sales

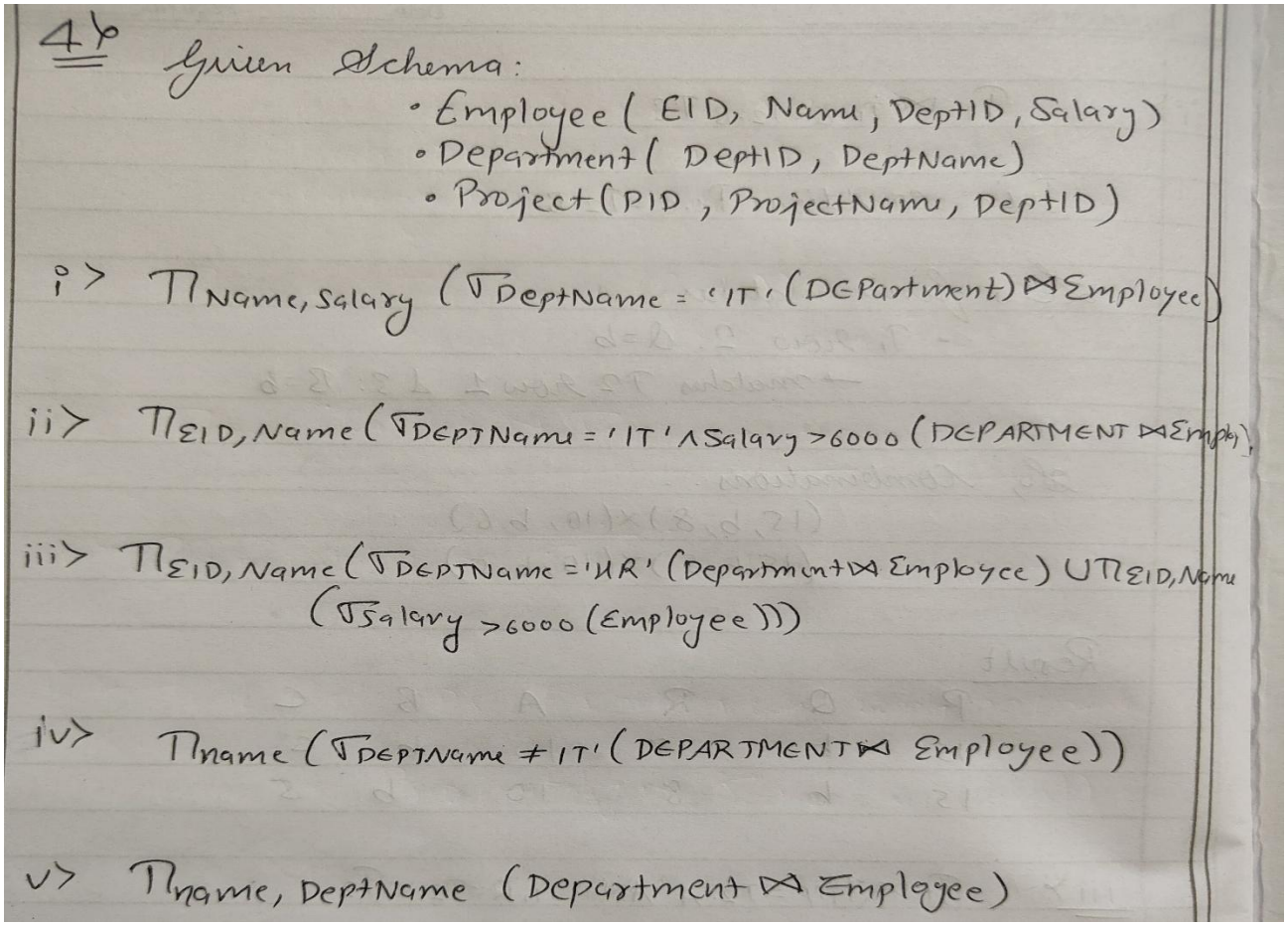
Project

PID	Project Name	DeptID
101	Project Alpha	10
102	Project Beta	20
103	Project Gamma	30

Write relational algebra expression for the following:

(i) Find the names and salaries of all employees in the 'IT' department.
(ii) Find the ID's and names of employees who are in the 'IT' department and have a salary greater than 6000.
(iii) Find the ID's and names of employees who are either in the 'HR' department or have a salary greater than 6000.
(iv) Find the names of employees who are not in the 'IT' department
(v) Find the names of employees along with their department names.

Ans:



5.

Given the relational tables:

Student:	
SID	Name
a	Alice
b	Bob
c	Carol

Project:	
PID	Project Name
p	Alpha
q	Beta
r	Gamma

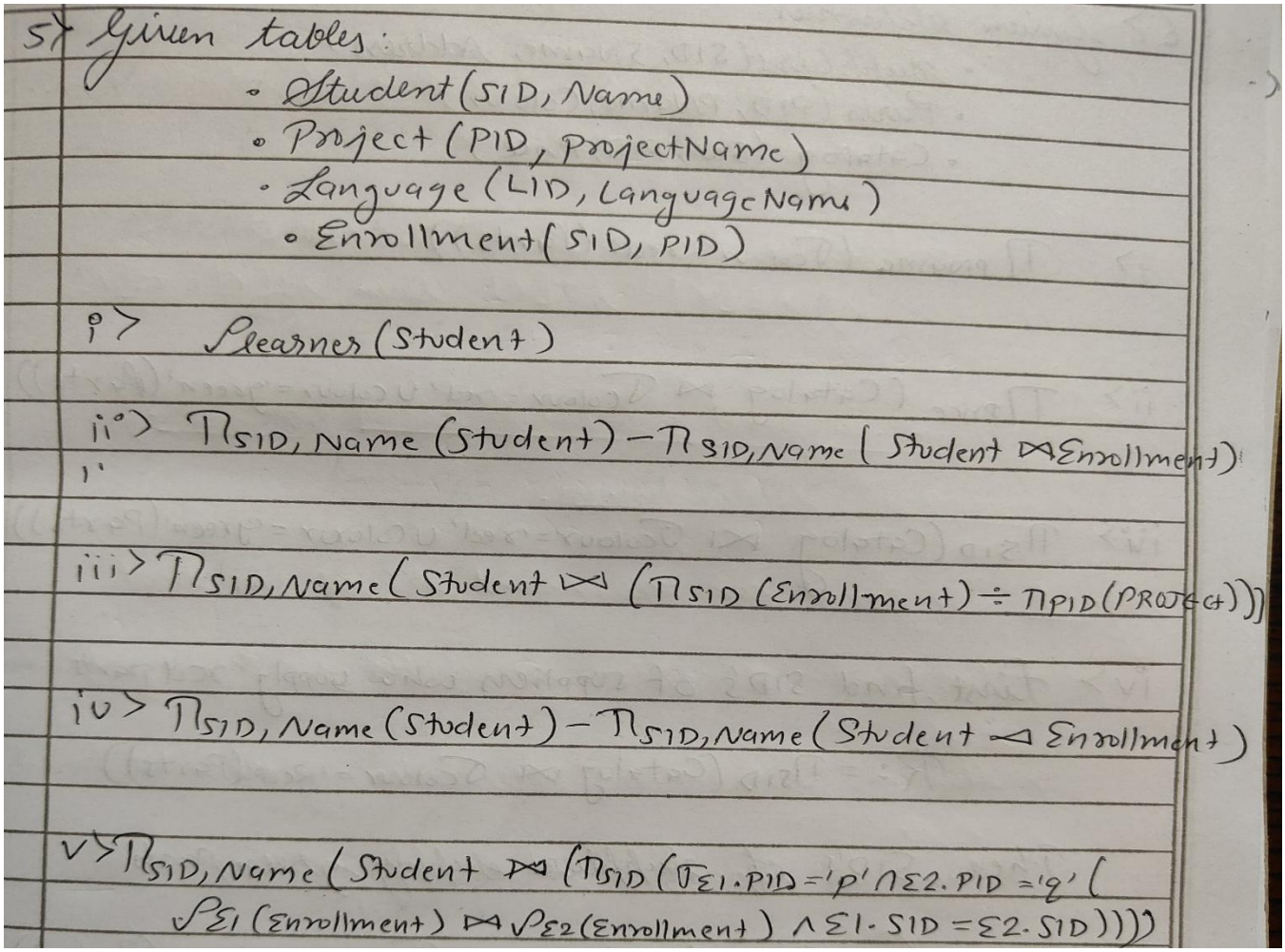
Language:	
LID	Language Name
x	Python
y	Java
z	C++

Enrollment:	
SID	PID
a	p
a	q
b	q
c	r

Write relational algebra expression for the following:

(i) Rename the student table to Learner and display it.
(ii) Find the students (learners) who are not enrolled in any project.
(iii) Find the students who are enrolled in all projects.
(iv) Find the students who are not enrolled in any project.
(v) Find the students who are enrolled in both the 'Alpha' and 'Beta' projects.

Ans:

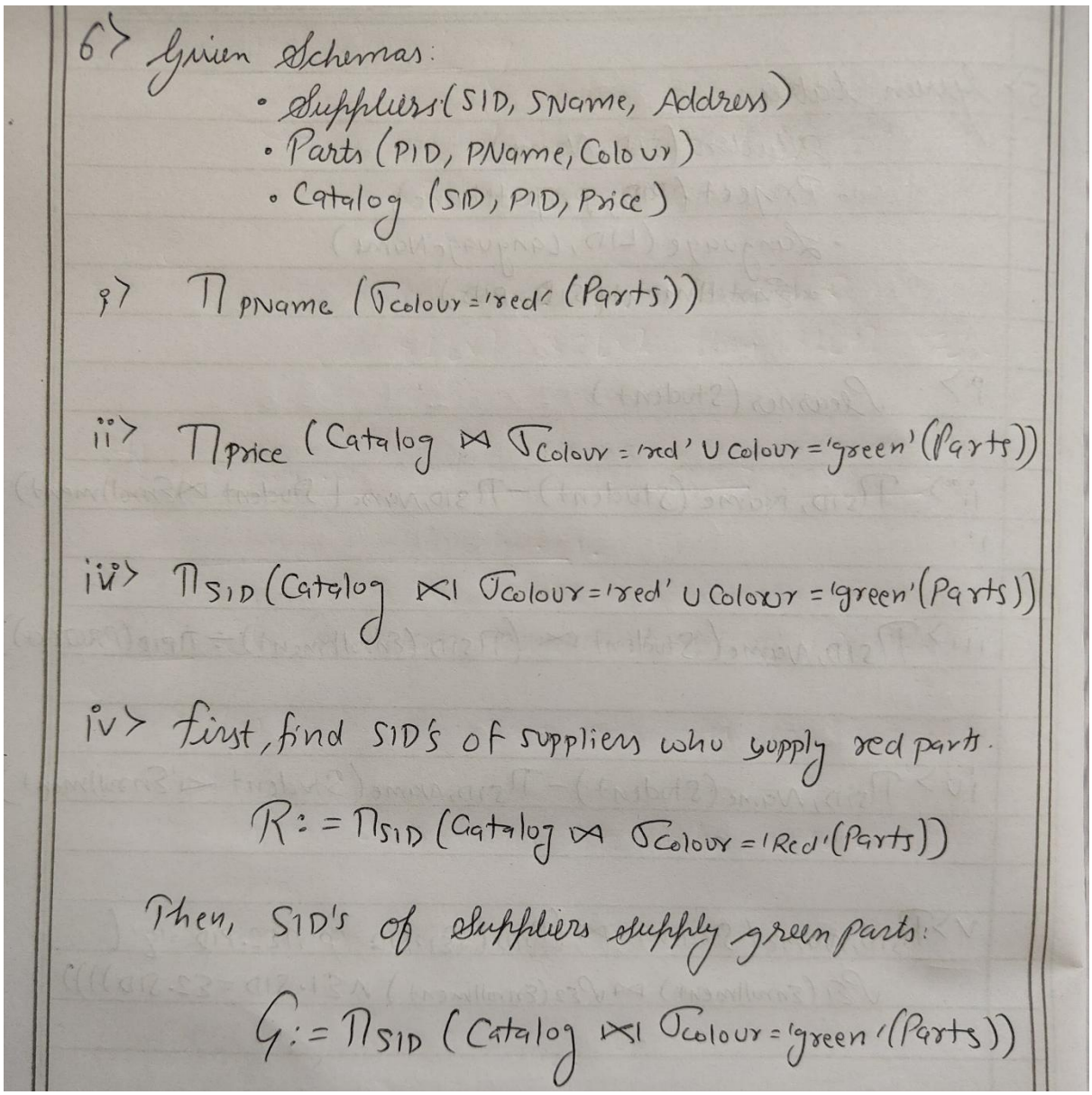


6.

Consider following schema :
Suppliers (SID , SName , address)
Parts (PID , PName , Colour)
Catalog (Sid , PID , Price)
Write relational algebra expression for following queries :

i) Find the names of all red parts.
ii) Find all prices for parts that were red or green.
iii) Find the SID's of all suppliers who supply part that is red or green.
iv) Find the SID's of all supplier who supply part that is red and green.

Ans:

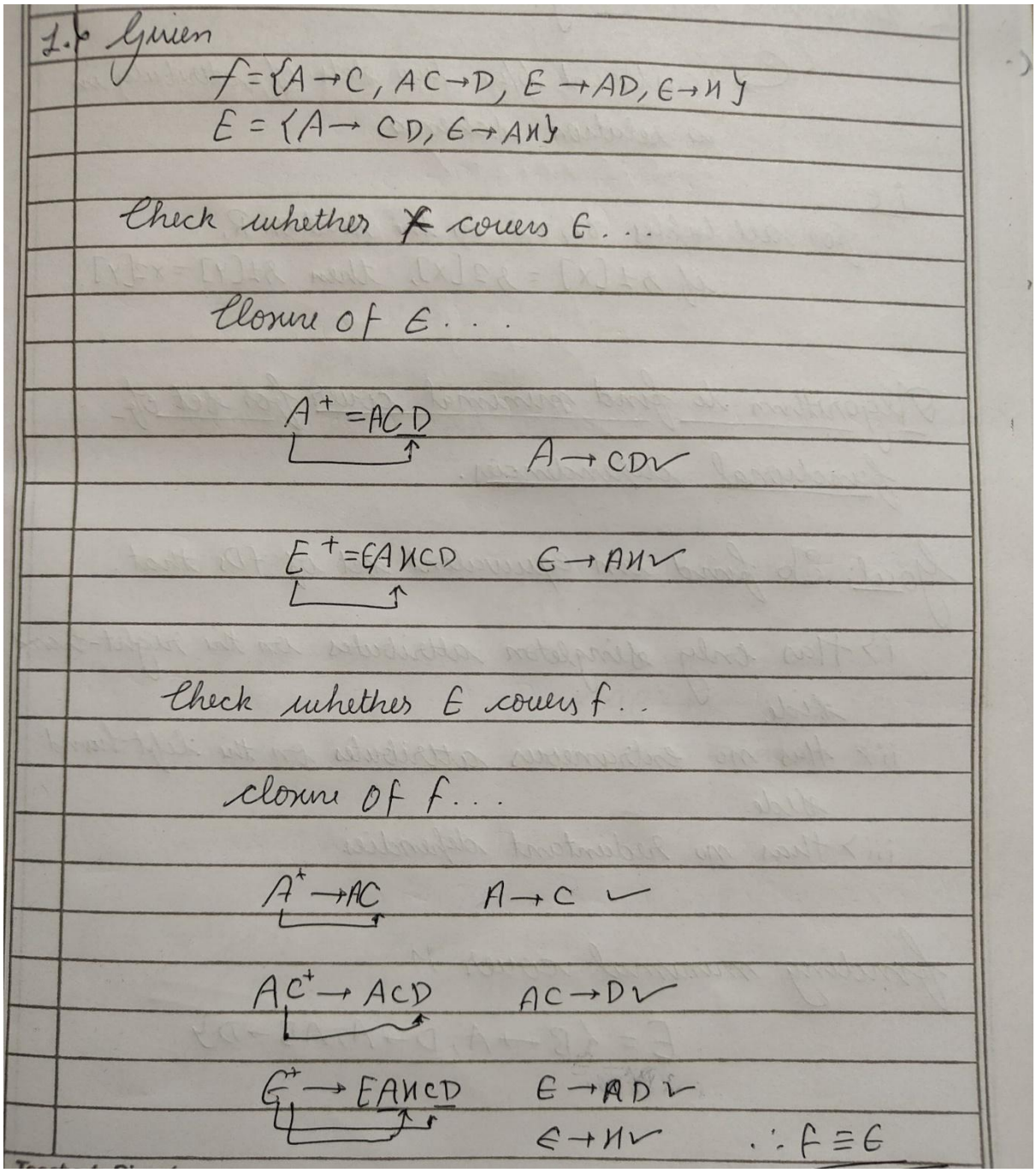


MODULE 3

1.

Consider two sets of functional dependency. F={A→C, AC→D, E→AD, E→H} E= {A→CD, E→AH}. Are they Equivalent?

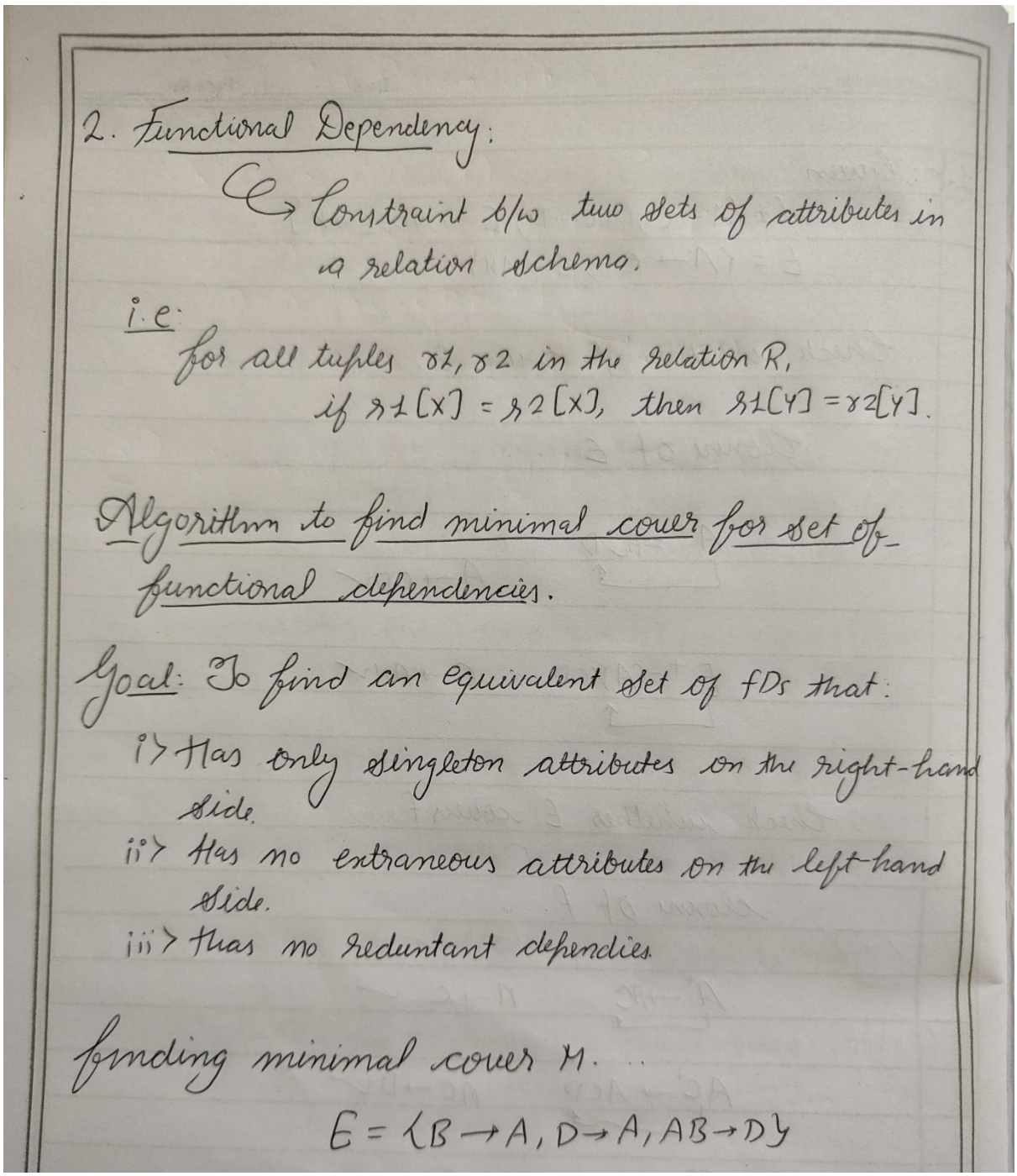
Ans:



2.

What is functional dependency? Write an algorithm to find minimal cover for set of functional dependencies. Construct minimal cover M for set of functional dependencies which are: E = {B → A, D → A, AB → D}

Ans:



2. Functional Dependency:

↳ Constraint b/w two sets of attributes in a relation schema.

i.e:

for all tuples $\gamma 1, \gamma 2$ in the relation R,

if $\gamma 1[X] = \gamma 2[X]$, then $\gamma 1[Y] = \gamma 2[Y]$.

Algorithm to find minimal cover for set of functional dependencies.

Goal: To find an equivalent set of FDs that:

i) Has only singleton attributes on the right-hand side.

ii) Has no extraneous attributes on the left-hand side.

iii) Has no redundant dependies.

finding minimal cover M...

$E = \{B \rightarrow A, D \rightarrow A, AB \rightarrow D\}$

#1> Make RHS singleton ✓

#2> Remove extraneous/Redundant functⁿ Dependency.

{B → A, D → A, AB → D}

$B^+ \rightarrow B$ (check closure of B from all other)

$D^+ \rightarrow D$ $AB^+ \rightarrow AB$

{B → A, D → A, AB → D}

#3> Check for Redundant f.D.

(Left → 1 attribute)

B → A, D → A,

Here D is not there in any A or B closure.

So, we can't reduce G.

AB → D

A is redundant.

$B^+ \rightarrow B$ (A)

So, if A is coming in B^+

Also, we can remove A.

E is already in its minimal cover.

E = {B → A, D → A, AB → D}

Lossless Join Decomposition

R (A, B, C, D, E, F)

f = {AB → C, C → D, D → EF, F → A, D → B}

D = {ABC, CDE, EF}

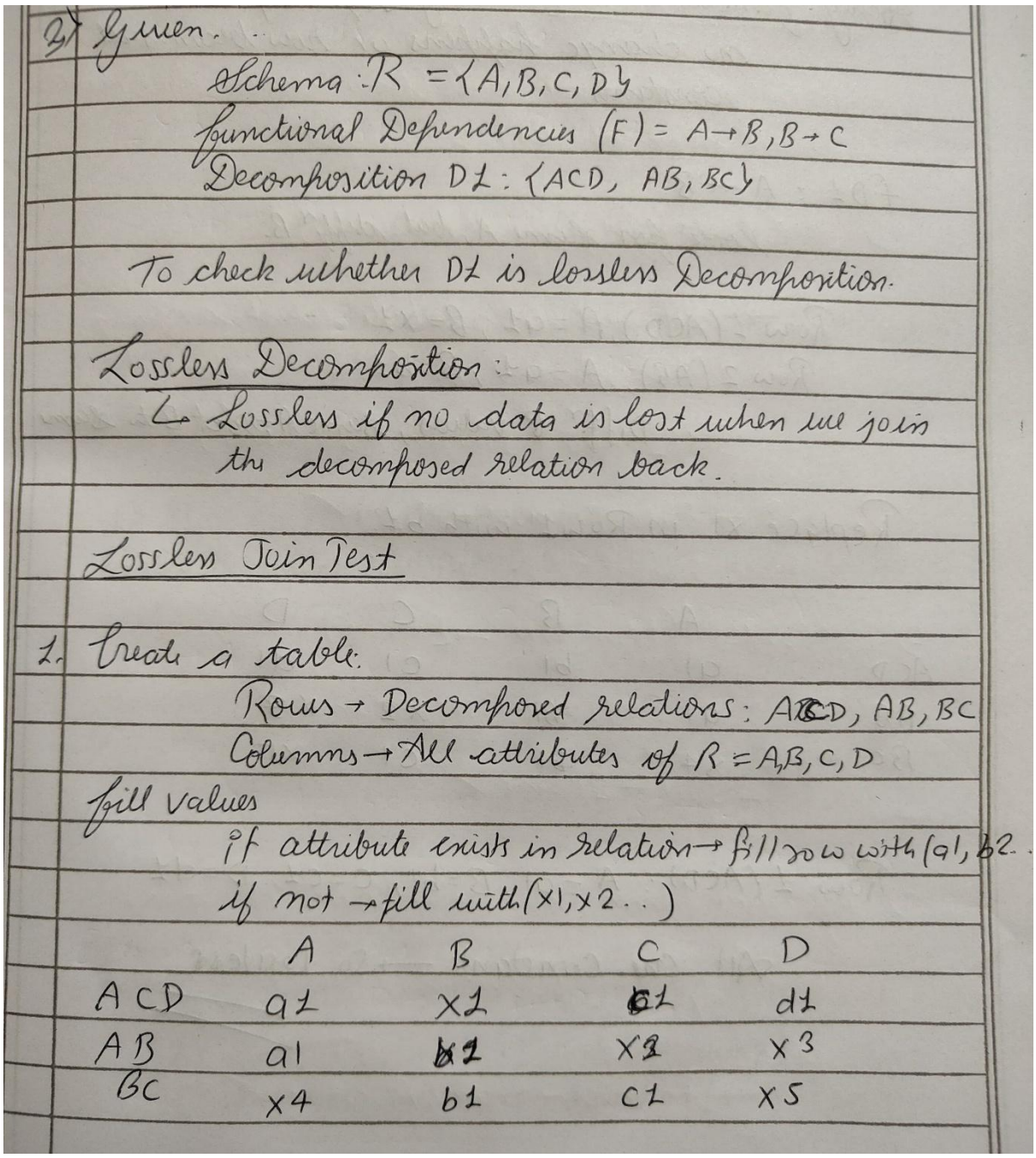
R1(ABC) R2(DE) R3(EF)

(1) Union of attr of all decomposed sub rel = attr of relation. ✓

3.

Consider the schema R = ABCD, subjected to FDs F = {A →B, B →C}, and the non-binary partition D1 = {ACD, AB, BC}. State whether D1 is a lossless decomposition? [give all steps in detail].

Ans:



3) Given.

Schema: $R = \{A, B, C, D\}$

Functional Dependencies $(F) = A \rightarrow B, B \rightarrow C$

Decomposition $D1: \{ACD, AB, BC\}$

To check whether D1 is lossless Decomposition.

Lossless Decomposition:

↳ Lossless if no data is lost when we join the decomposed relation back.

Lossless Join Test

1. Create a table:

Rows → Decomposed relations: ACD, AB, BC

Columns → All attributes of $R = A, B, C, D$

fill values

if attribute exists in relation → fill row with (a1, b2..

if not → fill with (x1, x2..)

	A	B	C	D
ACD	a1	x1	c1	d1
AB	a1	b1	x2	x3
BC	x4	b1	c1	x5

Apply functional dependency repeadly until no change happens or row becomes all constants.

fd1 : A → B.

look for same A, but diff^{r} B.

Row 1 (ACD); A = a1, B = x1

Row 2 (AB); A = a1, B = b1

Diff^{r} B values, same A ⇒ set B to same

Replace x1 in Row1 with b1.

	A	B	C	D
ACD	a1	b1	c1	d1
AB	a1	b1	x2	x3
BC	x4	b1	c1	x5

Row 1 (ACD): A = a1, B = b1, C = c1, D = d1

All are constants → So lossless.

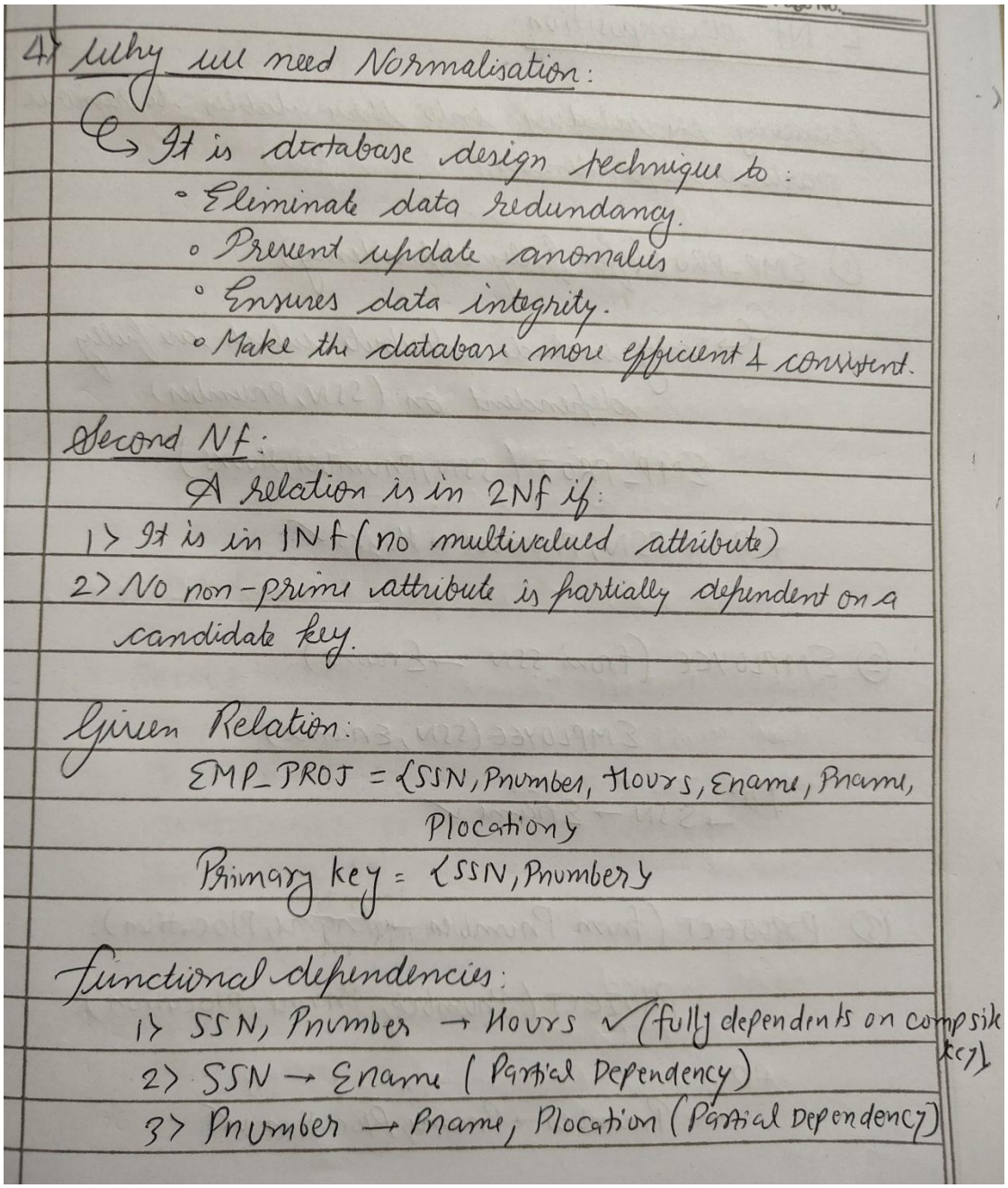
4.

What is the need for normalization? Explain 2nd normal form. Consider the relation EMP_PROJ = {SSn , Pnumber , Hours , Ename , Pname , Plocation}. Assume {SSn , Pnumber} as a primary key. The dependencies are

SSn ; Pnumber → {Hours}
SSn → {Ename}
Pnumber → {Pname, Plocation} ,

Normalize above relation into 2NF.

Ans:



2-Nf decomposition:

breaking the relation into three tables to remove partial dependencies...

① EMP_PROJ (for fully dependency):

↳ Only include attribute which are fully dependent on (SSN, Pnumber)

EMP_PROJ (SSN, Pnumber, Hours)

fD → SSN, Pnumber → Hours ✓

② EMPLOYEE (from SSN → Ename):

EMPLOYEE (SSN, Ename)

fD: → SSN → Ename ✓

③ PROJECT (from Pnumber → Pname, Plocation):

PROJECT (Pnumber, Pname, Plocation)

fD:

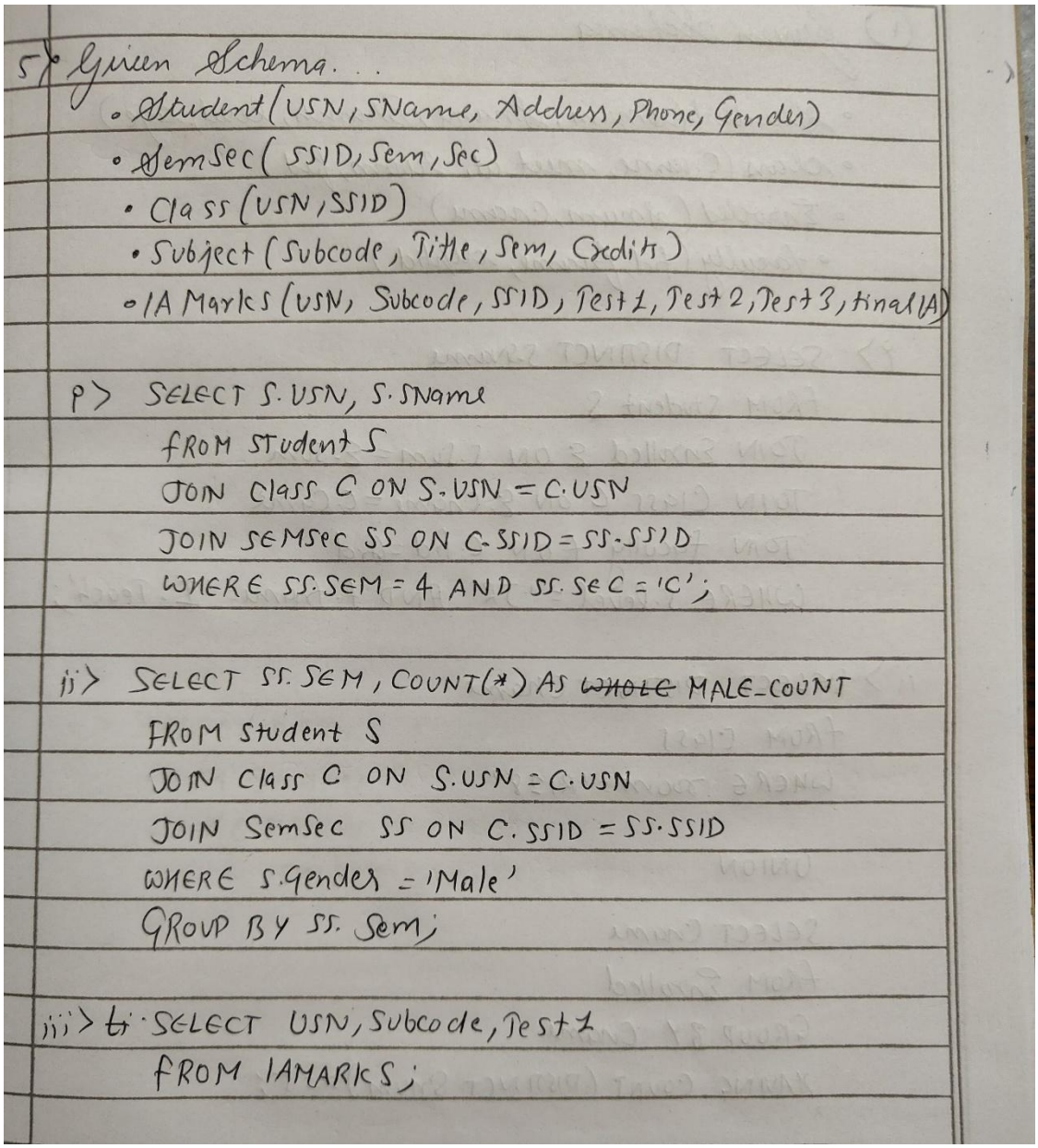
→ Pnumber → Pname, Plocation ✓

5.

Consider the schema for college database.
Student (USN , Sname , Address , Phone , Gender)
SemSec (SSID , Sem , Sec)
Class (USN , SSID)
Subject (Subcode , Title , Sem , Credits)
IAmarks (USN , Subcode , SSID , Test1 , Test2, Test3, Final IA)
Write SQL Query.

i) List all the students studying in 4th sem 'C' section.
ii) Compute total number of male students in each semester.
iii) List Test1 marks of all students in all subjects.

Ans:



5) Given Schema...
- Student (USN, SName, Address, Phone, Gender)
- SemSec (SSID, Sem, Sec)
- Class (USN, SSID)
- Subject (Subcode, Title, Sem, Credits)
- IA Marks (USN, Subcode, SSID, Test1, Test2, Test3, finalIA)

i) SELECT S.USN, S.SName
FROM Student S
JOIN Class C ON S.USN = C.USN
JOIN SEMSec SS ON C.SSID = SS.SSID
WHERE SS.SEM = 4 AND SS.SEC = 'C';

ii) SELECT SS.SEM, COUNT(*) AS ~~WHOLE~~ MALE_COUNT
FROM Student S
JOIN Class C ON S.USN = C.USN
JOIN SemSec SS ON C.SSID = SS.SSID
WHERE S.Gender = 'Male'
GROUP BY SS.Sem;

iii) SELECT USN, Subcode, Test1
FROM IAMARKS;

MODULE 4

1.

Consider the following relations:
Student(Snum, Sname, Branch, level, age)
Class(Cname, meet_at, room, fid)
Enrolled(Snum, Cname)
Faculty(fid, fname, deptid)
Write the following queries in SQL. No duplicates should be printed in any of the answers.

(i) Find the names of all Juniors (level = JR) who are enrolled in a class taught by I. Teach.

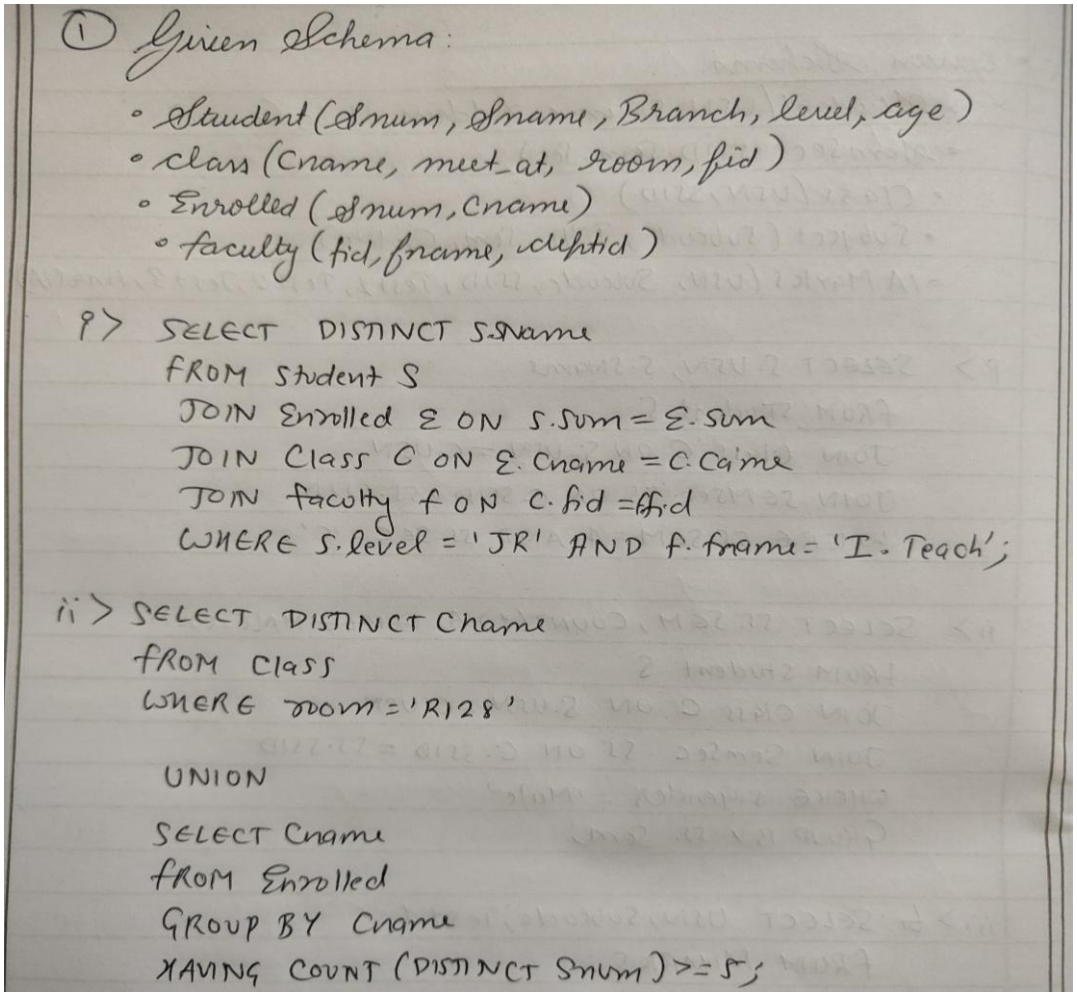
(ii) Find the names of all classes that either meet in room R128 or have five or more students enrolled.

(iii) For all levels except JR, print the level and rthe average age of students for that level.

(iv) For each faculty member that has taught classes only in room R128, print the faculty member's name and the total number of classes she or he has taught.

(v) Find the names of students not enrolled in any class.

Ans:



① Given Schema:

- Student (Snum, Sname, Branch, level, age)
- class (Cname, meet_at, room, fid)
- Enrolled (Snum, Cname)
- faculty (fid, fname, deptid)

i> SELECT DISTINCT S.Sname
FROM Student S
JOIN Enrolled E ON S.Snum = E.Snum
JOIN Class C ON E.Cname = C.Cname
JOIN faculty f ON C.fid = f.fid
WHERE S.level = 'JR' AND f.fname = 'I. Teach';

ii> SELECT DISTINCT Cname
FROM Class
WHERE room = 'R128'

UNION

SELECT Cname
FROM Enrolled
GROUP BY Cname
HAVING COUNT (DISTINCT Snum) >= 5;

iii> SELECT level, AVG(age) AS avg_age
FROM Student
WHERE level <> 'JR'
GROUP BY level;

iv> SELECT f.fname, COUNT(*) AS class_count
FROM faculty f
Join Class C ON f.fid = C.fid
GROUP BY f.fid, f.fname
HAVING COUNT(DISTINCT CASE WHEN C.room <> 'R128'
THEN 1 ELSE NULL END) = 0;

<> → Not equal to

v> SELECT DISTINCT S.Sname
FROM Student S
WHERE S.Snum NOT IN (
SELECT Snum FROM Enrolled
);

2.

Determine if the following schedule is serializable and explain your reasoning:

i) T1 : R(X)W(X) T2 : R(X)W(X) T1 : COMMIT T2 : COMMIT

ii) T1 : W(X)R(Y) T2 : R(X)W(Y) T1 : COMMIT T2 : COMMIT

Ans:

Definitions Recap:

- **Conflict Serializable**: A schedule is conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.
- **Precedence Graph**:
  - Nodes: Transactions (T1, T2)
  - Edges: If one transaction **writes or reads** a variable **before another** transaction **reads or writes** the same variable, draw an edge.

---

i)

Schedule:

T1: R(X) W(X)
T2: R(X) W(X)
T1: COMMIT
T2: COMMIT

Conflicts:

1. **T1 W(X)** before **T2 R(X)** → Write–Read conflict ⇒ T1 → T2
2. **T1 W(X)** before **T2 W(X)** → Write–Write conflict ⇒ T1 → T2

→ Both edges **T1 → T2**, no cycles.

Conclusion:

Serializable. Equivalent to serial order: **T1 → T2**

---

ii)

Schedule:

T1: W(X) R(Y)
T2: R(X) W(Y)
T1: COMMIT
T2: COMMIT

Conflicts:

1. **T1 W(X)** before **T2 R(X)** → Write–Read conflict ⇒ T1 → T2
2. **T1 R(Y)** before **T2 W(Y)** → Read–Write conflict ⇒ T1 → T2

→ Both edges are T1 → T2, no cycles

Conclusion:

Serializable. Equivalent to serial order: **T1 → T2**

3.

Consider the tables below:
Sailors (sid : integer, sname : string, rating : integer, age : real)
Boats (bid : integer, bname : string, color : string);
Reserves (sid : integer, bid : integer, day : date)
Write SQL queries for the following:

(i) Write create table statement for reserves.
(ii) Find all information of sailors who have reserved boat number 101.
(iii) Find the names of sailors who have reserved at least one boat.
(iv) Find the names of sailors who have reserved a red boat.
(v) Find the average age of sailors for each rating level.

Ans:

Sailors

(sid: integer, sname: string, rating: integer, age: real)

Boats

(bid: integer, bname: string, color: string)

Reserves

(sid: integer, bid: integer, day: date)

(i) Write the CREATE TABLE statement for Reserves

```
CREATE TABLE Reserves (
  sid INTEGER,
  bid INTEGER,
  day DATE,
  PRIMARY KEY (sid, bid, day),
```

```
 FOREIGN KEY (sid) REFERENCES Sailors(sid),
 FOREIGN KEY (bid) REFERENCES Boats(bid)
);
```

(ii) Find all information of sailors who have reserved boat number 101

```
SELECT DISTINCT S.*
FROM Sailors S
JOIN Reserves R ON S.sid = R.sid
WHERE R.bid = 101;
```

(iii) Find the names of sailors who have reserved at least one boat

```
SELECT DISTINCT S.sname
FROM Sailors S
JOIN Reserves R ON S.sid = R.sid;
```

(iv) Find the names of sailors who have reserved a red boat

```
SELECT DISTINCT S.sname
FROM Sailors S
JOIN Reserves R ON S.sid = R.sid
JOIN Boats B ON R.bid = B.bid
WHERE B.color = 'red';
```

(v) Find the average age of sailors for each rating level

```
SELECT rating, AVG(age) AS avg_age
FROM Sailors
GROUP BY rating;
```
